MySQL SQL Syntax Cheat Sheet

Exam-focused reference — syntax first, with the most important MySQL commands and patterns.

1. DATABASE COMMANDS

Command / Pattern	MySQL Syntax
CREATE DATABASE	CREATE DATABASE database_name;
CREATE IF NOT EXISTS	CREATE DATABASE IF NOT EXISTS database_name;
SELECT DATABASE	SELECT DATABASE();
USE DATABASE	USE database_name;
SHOW DATABASES	SHOW DATABASES;
DROP DATABASE	DROP DATABASE database_name;

2. TABLE — DDL

Command / Pattern	MySQL Syntax

CREATE TABLE	CREATE TABLE table_name (
	  column1 datatype [constraint],
	  column2 datatype [constraint],
	  ...
	);

CREATE TABLE IF NOT EXISTS	CREATE TABLE IF NOT EXISTS table_name (...);
SHOW TABLES	SHOW TABLES;
DESCRIBE / DESC	DESCRIBE table_name; -- or DESC table_name;
ALTER — ADD COLUMN	ALTER TABLE table_name ADD COLUMN column_name datatype;
ALTER — MODIFY COLUMN	ALTER TABLE table_name MODIFY COLUMN column_name new_datatype;
ALTER — RENAME COLUMN	ALTER TABLE table_name RENAME COLUMN old_name TO new_name;
ALTER — DROP COLUMN	ALTER TABLE table_name DROP COLUMN column_name;
ALTER — ADD CONSTRAINT	ALTER TABLE table_name ADD CONSTRAINT constraint_name ...;
ALTER — DROP PRIMARY KEY	ALTER TABLE table_name DROP PRIMARY KEY;
ALTER — DROP FOREIGN KEY	ALTER TABLE table_name DROP FOREIGN KEY constraint_name;
RENAME TABLE	RENAME TABLE old_name TO new_name;
TRUNCATE TABLE	TRUNCATE TABLE table_name;
DROP TABLE	DROP TABLE table_name;

3. CONSTRAINTS

Command / Pattern	MySQL Syntax
PRIMARY KEY	column_name datatype PRIMARY KEY
Composite PRIMARY KEY	PRIMARY KEY (column1, column2)
NOT NULL	column_name datatype NOT NULL
UNIQUE	column_name datatype UNIQUE
DEFAULT	column_name datatype DEFAULT value
CHECK	column_name datatype CHECK (condition)
FOREIGN KEY	FOREIGN KEY (column_name) REFERENCES parent_table(parent_column)

Command / Pattern	MySQL Syntax
Named constraint	CONSTRAINT constraint_name PRIMARY KEY (...) -- or UNIQUE / FOREIGN KEY / CHECK

4. INSERT — DML

Command / Pattern	MySQL Syntax
Insert one row	INSERT INTO table_name (col1, col2) VALUES (value1, value2);
Insert multiple rows	INSERT INTO table_name (col1, col2) VALUES (v1, v2), (v3, v4), (v5, v6);
Insert from SELECT	INSERT INTO table_name (col1, col2) SELECT col1, col2 FROM source_table;

5. SELECT — DQL

Command / Pattern	MySQL Syntax
Basic SELECT	SELECT column1, column2 FROM table_name;
All columns	SELECT * FROM table_name;
Alias	SELECT column_name AS alias_name FROM table_name;
DISTINCT	SELECT DISTINCT column_name FROM table_name;
WHERE	SELECT * FROM table_name WHERE condition;
ORDER BY	SELECT * FROM table_name ORDER BY column_name ASC;
Descending order	SELECT * FROM table_name ORDER BY column_name DESC;
LIMIT	SELECT * FROM table_name LIMIT number;
LIMIT + OFFSET	SELECT * FROM table_name LIMIT number OFFSET number;
BETWEEN	SELECT * FROM table_name WHERE column_name BETWEEN value1 AND value2;
IN	SELECT * FROM table_name WHERE column_name IN (value1, value2, value3);
NOT IN	SELECT * FROM table_name WHERE column_name NOT IN (value1, value2);
LIKE	SELECT * FROM table_name WHERE column_name LIKE 'pattern';
IS NULL	SELECT * FROM table_name WHERE column_name IS NULL;
IS NOT NULL	SELECT * FROM table_name WHERE column_name IS NOT NULL;
AND / OR / NOT	WHERE condition1 AND condition2 WHERE condition1 OR condition2 WHERE NOT condition

6. UPDATE / DELETE — DML

Command / Pattern	MySQL Syntax
UPDATE	UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
UPDATE using expression	UPDATE table_name SET salary = salary * 1.10 WHERE department = 'IT';

Command / Pattern	MySQL Syntax
DELETE selected rows	DELETE FROM table_name WHERE condition;
DELETE all rows	DELETE FROM table_name;

7. AGGREGATE + GROUPING

Command / Pattern	MySQL Syntax
COUNT	SELECT COUNT(*) FROM table_name;
COUNT(column)	SELECT COUNT(column_name) FROM table_name;
SUM	SELECT SUM(column_name) FROM table_name;
AVG	SELECT AVG(column_name) FROM table_name;
MIN / MAX	SELECT MIN(column_name), MAX(column_name) FROM table_name;
GROUP BY	SELECT group_column, COUNT(*) FROM table_name GROUP BY group_column;
HAVING	SELECT group_column, COUNT(*) FROM table_name GROUP BY group_column HAVING COUNT(*) > number;
GROUP BY + multiple aggregates	SELECT department, COUNT(*) AS emp_count, AVG(salary) AS avg_salary FROM employees GROUP BY department;

8. JOINS

Command / Pattern	MySQL Syntax
INNER JOIN	SELECT a.col, b.col FROM table_a a INNER JOIN table_b b ON a.key = b.key;
LEFT JOIN	SELECT a.col, b.col FROM table_a a LEFT JOIN table_b b ON a.key = b.key;
RIGHT JOIN	SELECT a.col, b.col FROM table_a a RIGHT JOIN table_b b ON a.key = b.key;
CROSS JOIN	SELECT * FROM table_a CROSS JOIN table_b;
SELF JOIN	SELECT a.col, b.col FROM employees a JOIN employees b ON a.manager_id = b.employee_id;
Multiple joins	SELECT ... FROM a JOIN b ON a.id = b.a_id JOIN c ON b.id = c.b_id;

9. SET OPERATIONS

Command / Pattern	MySQL Syntax
UNION	SELECT col FROM table_a UNION SELECT col FROM table_b;
UNION ALL	SELECT col FROM table_a UNION ALL SELECT col FROM table_b;

10. SUBQUERIES

Command / Pattern	MySQL Syntax
Scalar subquery	SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);
IN subquery	SELECT * FROM employees WHERE department_id IN (SELECT id FROM departments WHERE location = 'NY');
EXISTS	SELECT * FROM departments d WHERE EXISTS (SELECT 1 FROM employees e WHERE e.department_id = d.id);
Correlated subquery	SELECT * FROM employees e WHERE salary > (SELECT AVG(e2.salary) FROM employees e2 WHERE e2.department_id = e.department_id);

11. CASE

Command / Pattern	MySQL Syntax
Simple CASE	SELECT name, CASE department WHEN 'IT' THEN 'Technology' WHEN 'HR' THEN 'People' ELSE 'Other' END AS department_group FROM employees;
Searched CASE	SELECT name, CASE WHEN salary >= 80000 THEN 'High' WHEN salary >= 50000 THEN 'Medium' ELSE 'Low' END AS salary_band FROM employees;

12. NULL / CONDITIONAL FUNCTIONS

Command / Pattern	MySQL Syntax
COALESCE	SELECT COALESCE(column_name, replacement_value) FROM table_name;
IFNULL (MySQL)	SELECT IFNULL(column_name, replacement_value) FROM table_name;
NULLIF	SELECT NULLIF(expression1, expression2);
IF (MySQL)	SELECT IF(condition, value_if_true, value_if_false);

13. STRING FUNCTIONS

Command / Pattern	MySQL Syntax
CONCAT	SELECT CONCAT(first_name, ' ', last_name) FROM employees;
UPPER / LOWER	SELECT UPPER(name), LOWER(name) FROM employees;
LENGTH	SELECT LENGTH(name) FROM employees;
SUBSTRING	SELECT SUBSTRING(name, start, length) FROM employees;
TRIM	SELECT TRIM(column_name) FROM table_name;
REPLACE	SELECT REPLACE(column_name, 'old', 'new') FROM table_name;
LEFT / RIGHT	SELECT LEFT(name, 3), RIGHT(name, 3) FROM employees;

14. DATE / TIME FUNCTIONS

Command / Pattern	MySQL Syntax
Current date/time	SELECT CURDATE(); SELECT NOW();
Extract date part	SELECT YEAR(date_col), MONTH(date_col), DAY(date_col) FROM table_name;
DATE_FORMAT	SELECT DATE_FORMAT(date_col, '%Y-%m-%d') FROM table_name;
DATE_ADD	SELECT DATE_ADD(date_col, INTERVAL 7 DAY) FROM table_name;
DATEDIFF	SELECT DATEDIFF(end_date, start_date) FROM table_name;

15. WINDOW FUNCTIONS — MySQL 8+

Command / Pattern	MySQL Syntax
ROW_NUMBER	SELECT *, ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rn FROM employees;
RANK	SELECT *, RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk FROM employees;
DENSE_RANK	SELECT *, DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS drnk FROM employees;
Running total	SELECT order_date, amount, SUM(amount) OVER (ORDER BY order_date) AS running_total FROM sales;
LAG / LEAD	SELECT date_col, amount, LAG(amount) OVER (ORDER BY date_col) AS previous_amount, LEAD(amount) OVER (ORDER BY date_col) AS next_amount FROM sales;
Window aggregate	SELECT department, salary, AVG(salary) OVER (PARTITION BY department) AS dept_avg FROM employees;

16. VIEWS

Command / Pattern	MySQL Syntax
CREATE VIEW	CREATE VIEW view_name AS SELECT ...;
Replace view	CREATE OR REPLACE VIEW view_name AS SELECT ...;
Use view	SELECT * FROM view_name;
Drop view	DROP VIEW view_name;

17. INDEXES

Command / Pattern	MySQL Syntax
Create index	CREATE INDEX index_name ON table_name (column_name);
Unique index	CREATE UNIQUE INDEX index_name ON table_name (column_name);

Command / Pattern	MySQL Syntax
Composite index	CREATE INDEX index_name ON table_name (col1, col2);
Drop index	DROP INDEX index_name ON table_name;
Show indexes	SHOW INDEX FROM table_name;

18. TRANSACTIONS

Command / Pattern	MySQL Syntax
Start transaction	START TRANSACTION;
Commit	COMMIT;
Rollback	ROLLBACK;
Savepoint	SAVEPOINT savepoint_name;
Rollback to savepoint	ROLLBACK TO SAVEPOINT savepoint_name;
Release savepoint	RELEASE SAVEPOINT savepoint_name;

19. COMMON DATABASE OBJECT / METADATA COMMANDS

Command / Pattern	MySQL Syntax
Show table definition	SHOW CREATE TABLE table_name;
Show columns	SHOW COLUMNS FROM table_name;
Show indexes	SHOW INDEX FROM table_name;
Show databases	SHOW DATABASES;
Show tables	SHOW TABLES;

20. QUERY ORDER — VERY IMPORTANT

Command / Pattern	MySQL Syntax
Logical processing order	FROM / JOIN WHERE GROUP BY HAVING SELECT DISTINCT ORDER BY LIMIT
Typical writing order	SELECT ... FROM ... JOIN ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT ...;

High-Value Exam Memory

• **DDL:** CREATE, ALTER, DROP, TRUNCATE, RENAME → structure/objects.

• **DML:** INSERT, UPDATE, DELETE → modify data.

• **DQL:** SELECT → retrieve data.

• **WHERE:** filters rows before grouping.

• **GROUP BY:** creates groups.

• **HAVING:** filters groups, commonly using aggregates.

• **JOIN:** combines rows from tables using a relationship/condition.

• **UNION:** combines result sets and removes duplicates; UNION ALL keeps duplicates.

• **DELETE vs TRUNCATE vs DROP:** selected rows vs all rows/keep table vs remove table.

• **PARTITION BY:** divides rows into window-function groups; it does not collapse rows like GROUP BY.

• **ROW_NUMBER vs RANK vs DENSE_RANK:** unique sequence vs gaps after ties vs no gaps after ties.

• **MySQL:** window functions require MySQL 8.0+.